

Full Code Walkthrough

Every SQL Query and Python Script, Explained Block-by-Block

Inventory & Demand Forecasting Analytics for E-commerce Fulfillment

How to use this document: each file is broken into numbered blocks. Read the code, read the explanation directly below it, then try covering the explanation and re-explaining the block in your own words out loud — that's the actual skill an interviewer is testing, not recognition.

PART A — THE SQL LAYER

Four files, run in order against a SQLite database built from the CSV data. Each file builds SQL VIEWS (saved queries) that the next file/script can reference — this mirrors a real analytics stack where you build reusable data marts, not one-off ad hoc queries.

sql/02_demand_stats.sql

Computes trailing-90-day average and standard deviation of daily demand per SKU — the statistical inputs to the safety-stock formula.

Block 1 — collapse multi-warehouse rows into one row per SKU per day

```
CREATE VIEW v_daily_demand_by_sku AS
SELECT date, sku_id,
       SUM(demand_units) AS demand_units,
       SUM(sold_units) AS sold_units,
       SUM(loss_sales_units) AS loss_sales_units
FROM daily_sales
GROUP BY date, sku_id;
```

The raw table has one row per SKU PER WAREHOUSE per day. For network-wide demand statistics, we don't care which warehouse — we want total demand for the SKU across all 5 warehouses combined. SUM() + GROUP BY (date, sku_id) does exactly that collapse. This is the most basic and most important SQL pattern in the whole project: aggregate rows down to the grain you actually need.

Block 2 — the statistics, using a CTE (Common Table Expression)

```
WITH last_90 AS (
  SELECT * FROM v_daily_demand_by_sku
  WHERE date >= (SELECT date(MAX(date), '-89 day') FROM v_daily_demand_by_sku)
),
stats AS (
  SELECT sku_id, COUNT(*) AS days_observed,
         AVG(demand_units) AS avg_daily_demand,
         AVG(demand_units * demand_units) - AVG(demand_units) * AVG(demand_units) AS
variance_demand
  FROM last_90 GROUP BY sku_id
)
SELECT sku_id, days_observed,
       ROUND(avg_daily_demand, 2) AS avg_daily_demand,
       ROUND(SQRT(MAX(variance_demand, 0)), 2) AS stddev_daily_demand
FROM stats;
```

A CTE (the WITH ... AS block) is just a named, temporary query you can reference below it — used here to keep the logic readable in two clear steps instead of one nested mess. Step 1 (last_90) filters to the most recent 90 days. Step 2 (stats) computes the average, and variance using the identity $\text{Var}(X) = E[X^2] - (E[X])^2$ — SQLite has no built-in STDDEV(), so this is the manual formula. MAX(variance_demand, 0) guards against a tiny negative number from floating-point rounding before taking a square root (SQRT of a negative number would error).

Q: Why 90 days and not the full 2 years of data?

A: Demand drifts over time (there's a growth trend in the data). A 90-day window keeps the average responsive to *recent* behavior instead of being diluted by data from over a year ago. It's a deliberate tradeoff — shorter windows are more responsive but noisier; I picked 90 days as a reasonable middle ground and would tune it with real data.

Q: Why do you need standard deviation at all, why not just use the average?

A: The average tells you what typically happens; standard deviation tells you how much it varies. Safety stock exists specifically to protect against that variation — a SKU that always sells exactly 50 units/day needs zero safety stock, while a SKU that averages 50 but swings between 10 and 90 needs a lot, even though their averages are identical.

sql/03_abc_analysis.sql

Classifies every SKU into A/B/C tiers based on trailing-12-month revenue contribution, using window functions.

Block 1 — revenue per SKU

```
CREATE VIEW v_sku_revenue_12m AS
SELECT ds.sku_id, sm.category,
       SUM(ds.sold_units) AS units_sold_12m,
       SUM(ds.sold_units * sm.unit_price) AS revenue_12m,
       SUM(ds.sold_units * (sm.unit_price - sm.unit_cost)) AS gross_margin_12m
FROM daily_sales ds
JOIN sku_master sm ON sm.sku_id = ds.sku_id
WHERE ds.date >= (SELECT date(MAX(date), '-364 day') FROM daily_sales)
GROUP BY ds.sku_id, sm.category;
```

A standard JOIN + aggregation: bring in price/cost from the SKU master table, multiply units sold by price to get revenue (and by margin to get gross margin, a bonus metric). Notice revenue uses SOLD units, not DEMAND units — you can only recognize revenue on what actually sold, not on lost sales. This is a subtlety worth mentioning: ABC analysis based on sold units can slightly under-rank a SKU that's chronically stocked out, since its true demand-based revenue potential is higher than what's recorded.

Block 2 — the window functions (the important part)

```
RANK() OVER (ORDER BY revenue_12m DESC) AS revenue_rank,
SUM(revenue_12m) OVER (ORDER BY revenue_12m DESC
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_revenue,
SUM(revenue_12m) OVER () AS total_revenue
```

This is the single most important SQL concept in the project — window functions. Unlike GROUP BY (which collapses rows), a window function computes a value PER ROW while still looking across a defined set of other rows ("the window").

- RANK() OVER (ORDER BY revenue_12m DESC) — gives every SKU a rank, 1 = highest revenue, without collapsing any rows.
- SUM(...) OVER (ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) — a running total: for each row, sum the revenue of every row from the very top down to this one (in revenue-descending order). This is the literal mechanics of a Pareto/cumulative curve.
- SUM(...) OVER () — no ORDER BY, no ROWS clause = sum across ALL rows, i.e. grand total. Used as the denominator.

Cumulative % = running_revenue / total_revenue. This single line answers "what fraction of all revenue comes from the top N SKUs" for every N simultaneously.

Block 3 — the classification

```
CASE
  WHEN cumulative_pct_revenue <= 80 THEN 'A'
  WHEN cumulative_pct_revenue <= 95 THEN 'B'
  ELSE 'C'
END AS abc_class
```

A simple CASE statement (SQL's if/elif/else) applies the 80/95 Pareto cutoffs. Because cumulative_pct_revenue is monotonically increasing as rank increases, this cleanly buckets every SKU exactly once.

Q: What's the difference between RANK() and ROW_NUMBER()?

A: RANK() gives tied values the same rank and skips the next rank (1,1,3); ROW_NUMBER() gives every row a unique sequential number regardless of ties (1,2,3). I used RANK() since two SKUs with genuinely identical revenue should be treated as tied, not arbitrarily ordered.

Q: Could you do this without window functions?

A: Yes, with a self-join or subquery counting how many SKUs have revenue >= this SKU's revenue — but it's much less readable and typically slower, since the database can compute window functions in a single sorted pass instead of a row-by-row correlated subquery.

sql/04_reorder_points.sql

Combines demand stats + ABC class + lead time into the safety stock and reorder point per SKU, and compares against current stock to generate an action list.

Block 1 — the formula, with Z tied to ABC class

```
CASE a.abc_class WHEN 'A' THEN 2.05 WHEN 'B' THEN 1.65 ELSE 1.28 END AS service_level_z,
ROUND(
```

```

CASE a.abc_class WHEN 'A' THEN 2.05 WHEN 'B' THEN 1.65 ELSE 1.28 END
  * d.stddev_daily_demand * SQRT(sm.supplier_lead_time_days)
, 1) AS safety_stock_units,
ROUND(
  d.avg_daily_demand * sm.supplier_lead_time_days
  + [same Z formula] * d.stddev_daily_demand * SQRT(sm.supplier_lead_time_days)
, 1) AS reorder_point_units

```

A CASE statement picks the Z-value based on ABC class (this is a direct SQL implementation of the Z-score table from the theory — see the `LEARNING_GUIDE` for the derivation). Then it's a direct transcription of the two formulas: Safety Stock = $Z \times \sigma \times \sqrt{\text{lead time}}$; Reorder Point = (avg demand $\times$ lead time) + Safety Stock. The three tables joined together — `v_sku_demand_stats_90d` (avg/std), `v_abc_classification` (class), `sku_master` (lead time) — is a good example of a query needing multiple JOINS to assemble one business answer from normalized source tables.

Block 2 — turning the calculation into an action

```

CASE WHEN SUM(i.on_hand_units) <= r.reorder_point_units
  THEN 'REORDER NOW' ELSE 'OK' END AS action,
MAX(r.moq_units, ROUND(
  (r.avg_daily_demand * 21) - SUM(i.on_hand_units) + r.safety_stock_units
, 0)) AS suggested_order_qty

```

This is where the model becomes actionable instead of just descriptive: compare current on-hand stock (summed across warehouses) to the reorder point. If at or below it, flag REORDER NOW. The suggested quantity tops up to roughly 3 weeks of average demand plus the safety stock buffer, floored at the supplier's minimum order quantity (MOQ) — you can't order fewer units than a supplier will accept, so `MAX()` enforces that floor.

Q: Why compare NETWORK on-hand stock to a network-wide reorder point instead of per-warehouse?

A: Simplification for this project — real systems often do reorder points per warehouse since transfer costs between warehouses matter and a shortage in one FC can't always be filled from another's surplus quickly. I'd call this out explicitly as a next-step improvement if asked.

sql/05_fill_rate_impact.sql

Measures fill rate (the ultimate business metric) at the network level, by ABC class, and identifies the worst-performing SKUs.

```

SELECT
  ROUND(100.0 * SUM(sold_units) / NULLIF(SUM(demand_units), 0), 2) AS network_fill_rate_pct,
  ...
FROM daily_sales;

```

Straightforward ratio, with one defensive detail worth explaining if asked: `NULLIF(SUM(demand_units), 0)` converts a zero denominator into NULL instead of causing a divide-by-zero error — standard SQL defensive coding for any SKU that (hypothetically) had zero demand in the window.

The "worst-performing SKUs" query (`ORDER BY fill_rate_pct ASC LIMIT 15`) is a diagnostic query — in a real job, this is exactly the kind of query you'd run first when asked "why are we losing sales," to find where to focus investigation.

PART B — THE PYTHON LAYER

scripts/01 generate_data.py

Generates the synthetic dataset: 150 SKUs $\times$ 5 warehouses $\times$ 2 years of daily demand, with realistic seasonality and a deliberately naive baseline replenishment policy.

Block 1 — SKU master generation

```
categories = {
    "Mobiles & Accessories": {"n": 20, "price": (999, 25000), "cost_pct": 0.80, "base_demand": (8,
40)},
    ...
}
for cat, cfg in categories.items():
    for i in range(cfg["n"]):
        price = round(rng.uniform(*cfg["price"]), 2)
        cost = round(price * cfg["cost_pct"] * rng.uniform(0.92, 1.08), 2)
        lead_time = int(rng.choice([2,3,4,5,7,10,14], p=[0.15,0.2,0.2,0.15,0.15,0.1,0.05]))
        n_fcs = int(rng.choice([2, 3, 4], p=[0.3, 0.45, 0.25]))
```

A dictionary of category configs drives a loop that builds 150 SKU rows. Key Python patterns here: `rng.uniform(low, high)` draws a random float in a range (price, cost); `rng.choice([...], p=[...])` draws from a list with specified probabilities — used so lead times aren't uniformly random but skewed toward realistic values (more SKUs with short 2-5 day lead times, fewer with long 10-14 day ones), and so each SKU is stocked in 2-4 warehouses (not all 5), mimicking regional product availability.

Block 2 — the demand simulation loop

```
for day_idx, date in enumerate(dates):
    weekend_mult = 1.25 if dow >= 5 else 1.0
    promo_mult, is_promo = promo_multiplier(date)
    trend_mult = 1 + 0.15 * (day_idx / len(dates))
    for _, sku in sku_master.iterrows():
        for fc in fc_list:
            lam = max(per_fc_base * weekend_mult * promo_mult * trend_mult, 0.1)
            demand = rng.poisson(lam)
```

Nested loops: every day $\times$ every SKU $\times$ every assigned warehouse. The expected demand ("lam", short for the Poisson parameter lambda) is the SKU's base rate multiplied by three independent multipliers — weekend uplift, promo/festive multiplier, and a slow upward trend over the 2 years. `rng.poisson(lam)` then draws an actual random INTEGER count from the Poisson distribution with that mean — this is what makes demand a realistic, noisy count instead of a smooth deterministic curve.

Block 3 — stock tracking and the naive baseline policy

```
available = stock_state[key]
sold = min(demand, available)
lost = demand - sold
stock_state[key] = available - sold

if day_idx % REVIEW_CYCLE_DAYS == 0:
    on_order = sum(q for (_, q) in pending_orders[key])
    target_level = per_fc_base * (lt + REVIEW_CYCLE_DAYS)
    if stock_state[key] + on_order < target_level:
        order_qty = max(sku["moq_units"], round(target_level - stock_state[key] - on_order))
        pending_orders[key].append((day_idx + lt, order_qty))
```

This is the most important block in the file to be able to explain. `sold = min(demand, available)` caps actual sales at whatever stock exists — the gap (lost) is tracked separately, which is what later lets us compute a genuine fill rate. The reorder logic only runs every 7th day (`day_idx % REVIEW_CYCLE_DAYS == 0` — the modulo operator checks divisibility, a common Python/SQL idiom for "every Nth iteration"), and it targets only AVERAGE demand over the lead time + review cycle — no safety stock term at all. This is intentional: it's the realistic mistake of an unsophisticated ops process, and it's exactly the gap the rest of the project is built to close. `pending_orders` stores (arrival_day, quantity) tuples — the order doesn't arrive instantly, it arrives lead-time days later, modeling real supplier delay.

Q: Why simulate data instead of downloading a real dataset?

A: No public Flipkart-scale fulfillment dataset exists, and even if one did, using it wouldn't demonstrate anything different about my modeling skill. What matters is that the simulation **behaves** realistically — Poisson-distributed counts, genuine seasonality, and a believable naive baseline that produces real stockouts. I can defend every one of those choices, which is more valuable in an interview than an unexplainable real dataset would be.

Q: What's the % operator doing in `day_idx % REVIEW_CYCLE_DAYS == 0`?

A: Modulo — the remainder after division. `day_idx % 7 == 0` is true exactly every 7th day (0, 7, 14, 21...), which is how the weekly review cadence is implemented without a separate counter variable.

scripts/02_run_sql.py

Loads the CSVs into a real SQLite database, runs all four SQL files against it, and exports the resulting views to CSV for downstream use.

```
conn = sqlite3.connect("../ecommerce_fulfillment.db")
for f in sql_files:
    script = open(f).read()
    conn.executescript(script.split("-- Quick sanity check")[0] if "." in script else script)
demand_stats = pd.read_sql("SELECT * FROM v_sku_demand_stats_90d", conn)
demand_stats.to_csv(out + "sql_out_demand_stats.csv", index=False)
```

Two libraries doing two different jobs: `sqlite3` is Python's built-in SQL engine — `conn.executescript()` runs an entire multi-statement .sql file at once against the database. `pandas.read_sql()` then pulls a query's result directly into a `DataFrame`, which is the bridge between "data lives in SQL" and "data is usable in Python/Excel." This file is intentionally thin — it's SQL doing the analytical work, Python just orchestrating and exporting.

Q: Why SQLite instead of a real database server like PostgreSQL or MySQL?

A: SQLite is a serverless, file-based SQL database — perfect for a self-contained portfolio project since there's nothing to install or host, but it speaks standard SQL, so every query here would run on Postgres/MySQL/SQL Server with little to no modification. In a real company you'd obviously use whatever production database they run.

scripts/03_forecasting_model.py

Builds and benchmarks the ML demand forecasting model: feature engineering, causal (leakage-free) design, time-based evaluation, and model comparison.

Block 1 — weekly aggregation

```
sales["week_start"] = sales["date"].dt.to_period("W-MON").dt.start_time
weekly = (sales.groupby(["sku_id", "week_start"])
          .agg(demand_units=("demand_units", "sum"),
              is_promo_week=("is_promo_day", "max"))
          .reset_index())
```

`.dt.to_period("W-MON")` buckets every date into the Monday-starting week it belongs to — a standard pandas idiom for converting daily data to weekly. `groupby` + `agg` then sums demand within each SKU-week (and takes the MAX of the promo flag — if any day that week was a promo day, the whole week counts as a promo week).

Block 2 — causal feature engineering (avoiding data leakage)

```
def add_features(df):
    g = df.groupby("sku_id")["demand_units"]
    for lag in [1, 2, 4, 8]:
        df[f"lag_{lag}"] = g.shift(lag)
    df["roll_mean_4"] = g.shift(1).rolling(4).mean().reset_index(level=0, drop=True)
    df["roll_std_4"] = g.shift(1).rolling(4).std().reset_index(level=0, drop=True)
```

`g.shift(lag)` moves each SKU's demand series forward by "lag" weeks — `shift(1)` means "this row now holds LAST week's value," which is exactly what a lag feature needs to be. The critical detail: rolling averages are computed on `.shift(1)` FIRST, then `.rolling(4).mean()` — meaning the 4-week rolling average for week *t* only uses weeks (t-4) through (t-1), never week *t* itself. If you rolled BEFORE shifting, the "rolling average" for week *t* would include week *t*'s own actual demand — which the model would never have access to at real prediction time. This ordering is the single most important correctness detail in the whole ML pipeline.

Block 3 — the time-based split

```
cutoff = weekly["week_start"].max() - pd.Timedelta(weeks=8)
train = weekly[weekly["week_start"] <= cutoff]
test = weekly[weekly["week_start"] > cutoff]
```

Train on everything up to 8 weeks before the end of the dataset; test on those final 8 weeks. This mirrors real deployment — you always forecast forward in time using only the past. A random train/test split (common for many ML problems) would be wrong here, because it could put a week from the past in the test set while a later week from the same SKU sits in training, letting the model implicitly "see the future" through cross-SKU or seasonal patterns.

Block 4 — the WMAPE metric and model comparison

```
def wmape(y_true, y_pred):
    return np.sum(np.abs(y_true - y_pred)) / np.sum(np.abs(y_true)) * 100

rf = RandomForestRegressor(n_estimators=300, max_depth=12, min_samples_leaf=3, random_state=42)
rf.fit(X_train, y_train)
rf_pred = np.clip(rf.predict(X_test), 0, None)
```

WMAPE sums all absolute errors and divides by the sum of all actual values — volume-weighted, so it isn't distorted by a handful of tiny SKUs where a small absolute error looks like a huge percentage error. `RandomForestRegressor` is scikit-learn's implementation of the random forest algorithm; `n_estimators=300` means 300 individual decision trees are trained and averaged, `max_depth=12` limits how complex each tree can get (controls overfitting), and `min_samples_leaf=3` requires at least 3 training examples in every leaf node (also an overfitting guard — prevents trees from memorizing single data points). `np.clip(..., 0, None)` forces predictions to be non-negative, since negative predicted demand is meaningless.

Block 5 — feature importance and next-week forecast

```
importances = pd.Series(best_model.feature_importances_,
                        index=feature_cols).sort_values(ascending=False)
```

`feature_importances_` is an attribute scikit-learn's tree-based models expose automatically — it measures, on average across all 300 trees, how much each feature reduced prediction error when it was used to split the data. Higher = the model relied on it more. This is what let us report that last week's demand (`lag_1`) drives 57% of the model's decisions.

Q: What would happen if you rolled the mean BEFORE shifting, i.e. got the leakage wrong?

A: The model would appear far more accurate on the test set than it actually is, because it would effectively have access to the true answer (this week's demand) baked into a 'trend' feature. It would look great in your backtest and then perform much worse in real production, where that information genuinely isn't available yet. This is one of the most common and most dangerous mistakes in real-world forecasting projects.

Q: Why compare against a naive baseline and moving average instead of just reporting the ML model's accuracy alone?

A: A model's error number is meaningless in isolation — 14% WMAPE sounds fine or bad depending on what's achievable. Benchmarking against simple baselines proves the ML model is actually adding value (cutting error in half) rather than just being an impressive-sounding number.

scripts/04_ai_automation.py

Three automation components: anomaly detection (Isolation Forest), automated prioritized reorder alerts, and a before/after fill-rate simulation.

Block 1 — anomaly detection

```
roll_mean = grp["demand_units"].rolling(6, min_periods=3).mean()
grp["ratio_to_trend"] = grp["demand_units"] / roll_mean.replace(0, np.nan)
iso = IsolationForest(contamination=0.06, random_state=42, n_estimators=200)
grp["anomaly_flag"] = iso.fit_predict(grp[["ratio_to_trend"]])
```

Rather than feed raw demand into the anomaly detector directly, we first normalize it: `ratio_to_trend` = actual demand ÷ that SKU's own recent rolling average. A ratio of 1.0 means "exactly on trend"; 2.5 means "2.5x normal." This normalization matters because it makes anomalies comparable ACROSS SKUs of very different volume — without it, a naturally high-volume SKU's normal week would look more "extreme" in absolute terms than a low-volume SKU's actual anomaly. `IsolationForest.fit_predict()` returns -1 for detected anomalies and 1 for normal points; `contamination=0.06` tells the algorithm to expect roughly 6% of points to be anomalous, a tunable sensitivity knob.

Block 2 — urgency classification

```
def urgency(row):
    if not row["needs_reorder"]: return "OK"
    if row["days_of_cover"] < row["supplier_lead_time_days"] * 0.5: return "CRITICAL"
    if row["days_of_cover"] < row["supplier_lead_time_days"]: return "HIGH"
    return "MEDIUM"
```

`days_of_cover` (on-hand stock ÷ average daily demand) tells you how many days until you run out; `supplier_lead_time_days` tells you how many days until a new order would arrive if placed today. If `days_of_cover` is less than half the lead time, you will almost certainly stock out before a new order can possibly arrive — CRITICAL. This is a simple, explainable business rule layered on top of the numeric outputs — exactly the kind of translation from "raw number" to "actionable category" that makes an automated system usable by a non-technical ops team.

Block 3 — the before/after simulation

```
stock = (rp + ss) / n_fcs_for_sku # start topped up to policy target
for _, r in fc_grp.iterrows():
    arriving = [q for (a, q) in pending if a == day]
    if arriving: stock += sum(arriving); pending = [...]
    demand = r["demand_units"]
    sold = min(demand, stock)
    stock -= sold
    if stock <= (rp / n_fcs_for_sku) and not pending:
        pending.append((day + lt, order_qty))
```

This replays the exact same real historical daily demand (not synthetic new demand) but changes the POLICY: starting stock is topped up to the new reorder point + safety stock, and reordering is event-driven — it triggers the instant stock crosses the reorder point, rather than waiting for a slow weekly review. Everything else (the actual demand each day) is identical to what really happened. This isolates the effect of the POLICY CHANGE ALONE, which is what makes the resulting fill-rate comparison a fair, like-for-like estimate rather than a comparison against different underlying demand.

Q: Is your anomaly detection using a global threshold or is it per-SKU?

A: Per-SKU, by design — I normalize demand to each SKU's own trend before running Isolation Forest, specifically so a naturally low-volume SKU and a naturally high-volume SKU are judged on the same relative scale rather than one dominating just because of raw volume.

Q: Is the before/after result a real experiment?

A: No — it's a backtest simulation replaying real historical demand through a simulated policy, not a live A/B test. I say this proactively because understanding that distinction matters more than the headline number.

scripts/05_build_excel.py

Builds the Excel deliverable using openpyxl, with live formulas (not pasted values) for ABC classification and the reorder-point calculator.

Block 1 — styling conventions

```
INPUT_FONT = Font(name=FONT, color="0000FF", size=10)    # blue = hardcoded input
FORMULA_FONT = Font(name=FONT, color="000000", size=10)  # black = formula
```

A standard financial-modeling convention (used in real finance/analytics teams): blue text = a hardcoded input or external source, black text = a live calculated formula. This lets anyone auditing the workbook instantly tell what's raw data versus what's derived — important for trust in a model driving real purchasing decisions.

Block 2 — writing live formulas as strings

```
rank_cell = ws.cell(row=r, column=5, value=f"=RANK(D{r},$D$2:$D${last_row})")
cum_cell = ws.cell(row=r, column=6,
    value=f"=SUMPRODUCT(($E$2:$E${last_row}<=E{r})*$D$2:$D${last_row})/SUM($D$2:$D${last_row})")
class_cell = ws.cell(row=r, column=7, value=f'=IF(F{r}<=0.8,"A",IF(F{r}<=0.95,"B","C"))')
```

openpyxl lets you write a literal Excel formula as a Python string (starting with "=") into a cell — Excel/LibreOffice then evaluates it when the file is opened, exactly as if you'd typed it by hand. The f-string (f"...{r}...") injects the current row number so each row gets a correctly-addressed formula (row 2 references D2, row 3 references D3, etc.) — this is how 150 rows of live formulas get generated programmatically instead of typed one by one. \$D\$2:\$D\$151 (with dollar signs) is an ABSOLUTE reference — it won't shift when the formula is copied down, unlike a relative reference.

Block 3 — cross-sheet references, and why some values are hardcoded

```
ws.cell(row=r, column=2, value=f"=ABC_Analysis!G{r}").font = FORMULA_FONT
ws.cell(row=r, column=3, value=f"=Demand_Stats!C{r}").font = FORMULA_FONT
onh_cell = ws.cell(row=r, column=9, value=float(onhand_sorted.iloc[i]["on_hand_units"]))
onh_cell.font = INPUT_FONT
```

=ABC_Analysis!G{r} is a cross-sheet formula reference — it pulls the ABC class computed on a different tab. This is what makes the workbook a genuine cascading MODEL: change a value in Demand_Stats, and it flows through ABC_Analysis → Reorder_Point_Calc automatically. On-hand stock, by contrast, is written as a hardcoded value (INPUT_FONT, blue) — it's a raw system snapshot (what's actually sitting in the warehouse right now), not something derivable by a formula, so it's correctly treated as an input, not a calculation.

Block 4 — recalculation and verification

After building the workbook, it's run through a LibreOffice recalculation script (recalc.py) that opens the file, forces every formula to recompute, and reports any errors. This project's workbook came back with 0 errors across 1,809 formulas — worth stating explicitly, since it proves the formulas actually work rather than just looking plausible.

Q: Why write formulas as Python strings instead of just computing the values in Python and pasting numbers into Excel?

A: Pasting numbers would make the workbook a static snapshot — if a demand number changes, nothing recalculates and the analyst has to redo everything by hand or rerun the whole Python pipeline. Live formulas make it a real interactive model: change one input cell, and dependent cells cascade and update instantly. This is exactly what the resume line "built using Excel" should mean — a working spreadsheet model, not a spreadsheet-shaped report.

Q: What's the difference between \$D\$2 and D2 in a formula?

A: The dollar signs freeze that reference so it doesn't shift when the formula is copied to another cell — essential when the same range (like the full revenue column for a RANK comparison) needs to be referenced identically from every row.

Self-Test: Explain These Cold

Cover the code above and see if you can produce these from memory — this is the actual bar for "can explain in an interview," not just recognizing the code when you see it.

- Why does the CTE in 02_demand_stats.sql compute variance using $\text{AVG}(x^2) - \text{AVG}(x)^2$, and why guard it with $\text{MAX}(\dots, 0)$?
- Explain, step by step, what `SUM(revenue_12m) OVER (ORDER BY revenue_12m DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` computes.
- Why is the reorder trigger in the baseline policy (01_generate_data.py) missing a safety-stock term, and why does that matter?
- Walk through why rolling means must be computed on `.shift(1)`, not raw demand, in the forecasting feature engineering.
- Why is a time-based train/test split used instead of a random split?
- What does `contamination=0.06` control in the Isolation Forest, and why normalize demand to `ratio_to_trend` first?
- What's the difference between blue and black text in the Excel workbook, and why does that convention matter?
- Explain what `$D$2:$D$151` vs `D2` means in an Excel formula, and why it matters when generating 150 rows of formulas programmatically.